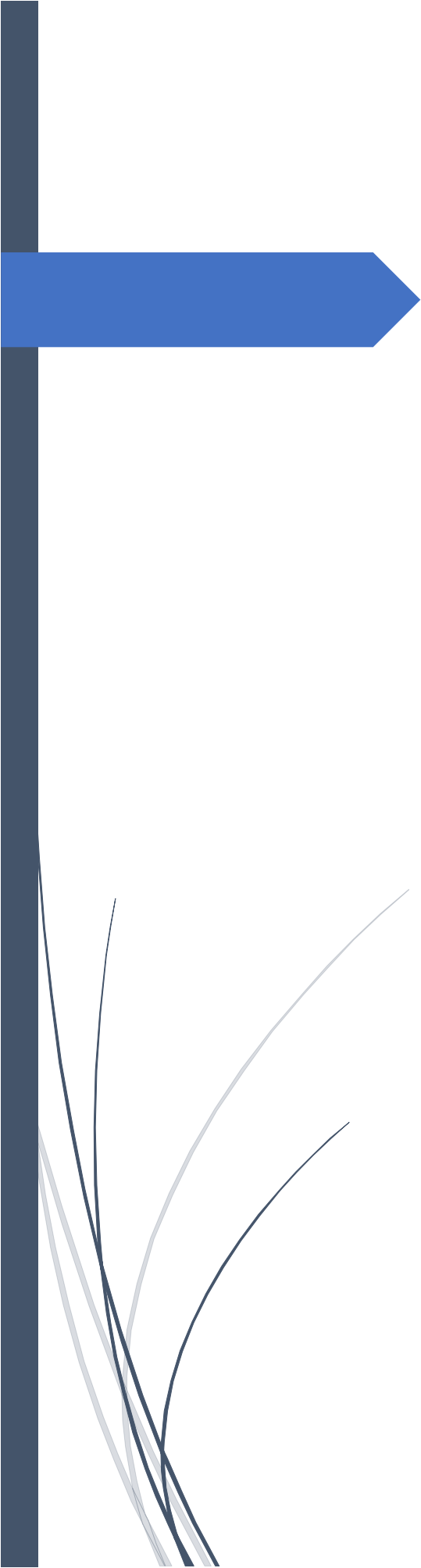




Power BI Advanced

Rupankar Ghosh

Power BI Advanced Course – Class 1

Data Modeling (Step-by-Step in Extreme Detail)

Duration: 2–3 Hours

Class Objective:

By the end of this class, you should be able to:

- Understand Power BI architecture.
- Import data correctly.
- Design a professional data model.
- Create a Star Schema.
- Build relationships properly.
- Create and use a Date Table.
- Apply data modeling best practices.

Part 1: What is Data Modeling?

Definition

Data modeling is the process of organizing tables and defining relationships so Power BI can analyze data accurately and efficiently.

Think of it as creating a **map** that tells Power BI how different tables are connected.

Example

Imagine a shop with four tables:

Sales

SaleID	ProductID	CustomerID	Date	Amount
1	101	C001	1-Jan	100

Products

ProductID	ProductName	Category
101	Laptop	Electronics

Customers

CustomerID	CustomerName	City
C001	Rahul	Kolkata

Date

Date	Month	Year
------	-------	------

The **Sales** table is connected to the other tables using common keys.

Part 2: Types of Tables

1. Fact Table

A Fact Table contains:

- Transactions
- Numeric values
- Foreign Keys

Example:

SaleID	ProductID	CustomerID	Date	Sales
1	101	C001	1-Jan	1200

Characteristics:

- Large table
- Many rows
- Contains measures like Sales, Profit, Quantity

Examples:

- Sales
- Orders
- Transactions
- Attendance

2. Dimension Table

Contains descriptive information.

Example:

Product Table

ProductID	ProductName	Brand
101	Laptop	Dell

Customer Table

CustomerID	CustomerName	City
------------	--------------	------

Date Table

Date	Month	Year
------	-------	------

Characteristics:

- Small table
- Descriptive columns
- Used for slicing and filtering

Interview Question

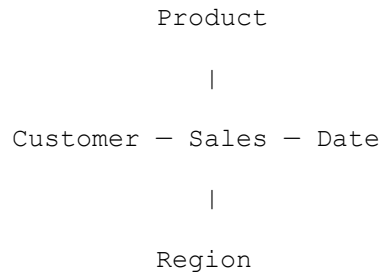
How do you identify a Fact Table?

Answer:

- Contains numerical values.
- Contains foreign keys.
- Usually has the largest number of rows.
- Records transactions or events.

Part 3: Star Schema

A Star Schema is the recommended model in Power BI.

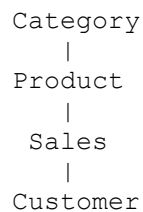


The **Sales** table is in the center, with dimension tables connected around it.

Advantages

- Fast performance
- Simple DAX
- Easy maintenance
- Better compression
- Cleaner reports

Part 4: Snowflake Schema



Dimension tables are connected to other dimension tables.

Problems

- Slower queries
- More relationships
- Complex DAX
- Harder to maintain

Interview Answer

Which schema is preferred in Power BI?

Answer: Star Schema, because it offers better performance and simpler modeling.

Part 5: Primary Key and Foreign Key

Primary Key

A column with **unique values**.

Example:

ProductID

101

102

103

No duplicates.

Foreign Key

A column that references a primary key in another table.

Example (Sales):

ProductID

101

101

102

Duplicates are allowed.

Part 6: Relationships

Open **Model View** to create relationships.

One-to-Many (1:*)

Most common relationship.

Products (1)

|
 |
Sales (*)

One product can appear in many sales.

Example

Products

ProductID

101

Sales

ProductID

101

101

101

Many-to-One (*:1)

Same relationship viewed from the other table.

One-to-One (1:1)

Both tables contain unique values.

Example:

- Employee
- Employee Details

Rarely used.

Many-to-Many (:)

Example:

Students

Courses

One student can take many courses, and one course can have many students.

Avoid unless necessary, because it can create ambiguity and affect calculations.

Part 7: Cardinality

Power BI detects:

- One-to-One
- One-to-Many
- Many-to-One
- Many-to-Many

Always verify that Power BI detected the correct cardinality.

Part 8: Cross Filter Direction

Single Direction (Recommended)

Products
↓
Sales

Filters flow from the dimension table to the fact table.

Advantages

- Better performance
 - Easier to troubleshoot
 - Recommended in most models
-

Both Direction

Filters move both ways.

Use only when required, such as some many-to-many scenarios or specific reporting needs, because it can introduce ambiguous filter paths.

Part 9: Active vs Inactive Relationships

Example:

Sales table contains:

- Order Date
- Ship Date

Date table contains:

- Date

Only one relationship can be active at a time.

To use the inactive relationship in a measure:

```
Sales by Ship Date =  
CALCULATE(  
    SUM(Sales[Sales]),  
    USERELATIONSHIP(Sales[ShipDate], Date[Date])  
)
```

Part 10: Date Table

Why create a Date Table?

Time intelligence functions such as:

- YTD
- MTD
- QTD
- YOY

require a proper date table.

Create one:

Date =

```
CALENDAR (  
    DATE (2023,1,1) ,  
    DATE (2025,12,31)  
)
```

Add columns:

```
Year = YEAR(Date[Date])
```

```
Month = FORMAT(Date[Date], "MMMM")
```

```
Month No = MONTH(Date[Date])
```

```
Quarter = "Q" & FORMAT(Date[Date], "Q")
```

Then:

1. Select the Date table.
2. Go to **Table tools**.
3. Choose **Mark as Date Table**.
4. Select the **Date** column.

Part 11: Hide Unnecessary Columns

Hide:

- ProductID
- CustomerID
- Foreign keys
- Technical columns

This keeps the Fields pane clean for report authors.

Part 12: Sort by Column

Problem:

Months sort alphabetically.

```
April  
August  
December  
February
```

Solution:

1. Create `Month No.`
2. Select the `Month` column.
3. Choose **Sort by Column** → `Month No.`

Now the months appear in calendar order.

Part 13: Model View Best Practices

- Use a Star Schema.
 - Keep only one fact table where possible.
 - Hide technical columns.
 - Rename tables and columns clearly.
 - Avoid duplicate relationships.
 - Use meaningful names for measures.
 - Create folders for measures.
 - Keep the model clean and organized.
-

Hands-on Practice (30–40 Minutes)

Use a Sales dataset with these tables:

- Sales
- Product
- Customer
- Date
- Region

Tasks:

1. Load all tables.
2. Identify the Fact table and Dimension tables.
3. Create one-to-many relationships.
4. Set cross-filter direction to **Single** where appropriate.
5. Create a Date table.
6. Mark it as the Date table.
7. Hide foreign key columns.
8. Sort Month by Month Number.
9. Verify the model follows a Star Schema.

Homework

1. Explain the difference between a Fact Table and a Dimension Table.
2. Compare Star Schema and Snowflake Schema.
3. Explain all four relationship types with examples.
4. Create a Date table using `CALENDAR()`.
5. Build a Star Schema using a sample Sales dataset.
6. Research when to use `USERELATIONSHIP()` and prepare one example.

In the next class, we'll cover **Measures, Calculated Columns, Calculated Tables, Evaluation Context (Row vs Filter Context), and the essential DAX functions that every Power BI developer should master.**

Power BI Advanced Course – Class 2

DAX Fundamentals to Intermediate (Extreme Detail)

Duration: 2.5–3 Hours

Learning Objectives

By the end of this class, you will understand:

- What DAX is
 - Measures vs Calculated Columns vs Calculated Tables
 - Row Context vs Filter Context (introduction)
 - Implicit vs Explicit Measures
 - Variables (`VAR`, `RETURN`)
 - Important aggregation, logical, and information functions
 - DAX best practices
-

Part 1: What is DAX?

DAX (Data Analysis Expressions) is the formula language used in Power BI, Power Pivot, and SSAS Tabular.

DAX is used to:

- Perform calculations
- Create measures
- Create calculated columns
- Create calculated tables
- Build KPIs
- Perform time intelligence
- Write business logic

Think of DAX as **Excel formulas + SQL logic + Business Intelligence**.

Part 2: DAX Components

DAX mainly consists of:

1. Functions
2. Operators
3. Constants
4. References
5. Variables

Example:

```
Total Sales =  
SUM(Sales[SalesAmount])
```

- SUM → Function
- Sales → Table
- SalesAmount → Column

Part 3: Measures vs Calculated Columns vs Calculated Tables

This is one of the most common interview questions.

1. Measure

A measure is calculated **only when needed** (at query time).

Example:

```
Total Sales =  
SUM(Sales[SalesAmount])
```

If a report has:

- 10 visuals → measure is evaluated for those visuals.
- 100 visuals → evaluated as required.

Characteristics

- Does **not** store values.
- Fast.
- Uses less memory.
- Dynamic based on filters and slicers.

Use measures for:

- Sales
- Profit
- Margin %
- KPIs
- YTD
- YOY

2. Calculated Column

A calculated column is computed when the data is refreshed and the result is stored in the model.

Example:

```
Profit =  
Sales[SalesAmount] - Sales[Cost]
```

Every row gets a stored value.

Characteristics

- Increases model size.
- Uses memory.
- Doesn't automatically recalculate with report filters.

Use calculated columns for:

- Category labels
- Age groups
- Profit per row
- Custom keys

3. Calculated Table

Creates a new table using DAX.

Example:

```
TopCustomers =  
FILTER(  
    Customer,
```

```
Customer[Sales] > 100000
)
```

Used for:

- Virtual dimensions
- Summary tables
- Bridge tables
- Custom calendars

Comparison

Feature	Measure	Calculated Column	Calculated Table
Stored in memory	✗	✓	✓
Dynamic	✓	✗	Refresh only
Uses filters	✓	Limited	Refresh only
Best for	KPIs	Row calculations	New tables

Part 4: Implicit vs Explicit Measures

Implicit Measure

Drag a numeric field into a visual.

Power BI creates:

```
SUM(Sales[SalesAmount])
```

Automatically.

Not recommended for professional reports.

Explicit Measure

You create it yourself.

```
Total Sales =  
SUM(Sales[SalesAmount])
```


Recommended because:

- Reusable
 - Easier to maintain
 - Better naming
 - Supports advanced DAX
-

Part 5: Aggregation Functions

SUM()

Adds all numbers.

```
Total Sales =  
SUM(Sales[SalesAmount])
```

AVERAGE()

```
Average Sales =  
AVERAGE(Sales[SalesAmount])
```

MIN()

```
Minimum Sales =  
MIN(Sales[SalesAmount])
```

MAX()

```
Maximum Sales =  
MAX(Sales[SalesAmount])
```

COUNT()

Counts non-blank values.

```
COUNT(Sales[OrderID])
```

COUNTA()

Counts all non-empty values, including text.

```
COUNTA (Customer [CustomerName])
```

COUNTROWS()

Counts rows in a table.

```
COUNTROWS (Sales)
```

This is generally preferred over `COUNT ()` when counting records.

DISTINCTCOUNT()

Counts unique values.

```
Customers =  
DISTINCTCOUNT (Sales [CustomerID])
```

Part 6: Mathematical Functions

DIVIDE()

Preferred over `/`.

Instead of:

```
Margin =  
SUM (Sales [Profit]) / SUM (Sales [SalesAmount])
```

Use:

```
Margin % =  
DIVIDE (  
    SUM (Sales [Profit]),  
    SUM (Sales [SalesAmount]),  
    0  
)
```

The third argument (0) is returned if the denominator is zero, avoiding errors.

ABS()

```
ABS (-500)
```

Returns:

```
500
```

ROUND()

```
ROUND (25.678, 2)
```

Result:

```
25.68
```

Part 7: Logical Functions

IF()

```
Target =  
IF (  
    [Total Sales] > 500000,  
    "Achieved",  
    "Not Achieved"  
)
```

Nested IF()

```
Grade =  
IF (  
    Sales[Marks] >= 80,  
    "A",  
    IF (  
        Sales[Marks] >= 60,  
        "B",  
        "C"  
    )  
)
```

Works, but becomes difficult to maintain.

SWITCH()

Cleaner than multiple IF statements.

```
Grade =  
SWITCH(  
    TRUE(),  
    Sales[Marks] >=80, "A",  
    Sales[Marks] >=60, "B",  
    Sales[Marks] >=40, "C",  
    "Fail"  
)
```

This is the preferred approach for multiple conditions.

Part 8: Variables (VAR & RETURN)

Instead of repeating calculations:

```
Profit Margin =  
DIVIDE(  
    SUM(Sales[Profit]),  
    SUM(Sales[SalesAmount])  
)
```

Write:

```
Profit Margin =  
VAR SalesValue =  
    SUM(Sales[SalesAmount])  
  
VAR ProfitValue =  
    SUM(Sales[Profit])  
  
RETURN  
DIVIDE(  
    ProfitValue,  
    SalesValue  
)
```

Benefits

- Faster execution

- Cleaner code
- Easier debugging
- Improves readability

Use `VAR` whenever a value is reused.

Part 9: Information Functions

SELECTEDVALUE()

Returns the selected value if exactly one value is selected.

```
Selected Region =  
SELECTEDVALUE (Customer[Region])
```

If one region is selected:

```
East
```

If multiple regions are selected:

```
Blank
```

With a default value:

```
Selected Region =  
SELECTEDVALUE (  
Customer[Region],  
"Multiple Regions"  
)
```

HASONEVALUE()

Checks if exactly one value is selected.

```
IF (  
HASONEVALUE (Customer[Region]),  
"One Region",  
"Multiple Regions"  
)
```

ISFILTERED()

Checks whether a column is filtered.

```
IF (
  ISFILTERED (Customer[City]),
  "Filtered",
  "Not Filtered"
)
```

Part 10: Text Functions

CONCATENATE()

```
Full Name =
CONCATENATE (
  Customer[FirstName],
  Customer[LastName]
)
```

A more flexible option is:

```
Full Name =
Customer[FirstName] & " " & Customer[LastName]
```

LEFT()

```
LEFT (Customer [Name], 3)
```

RIGHT()

```
RIGHT (Customer [Name], 2)
```

MID()

```
MID (Customer [Name], 2, 5)
```

LEN()

```
LEN (Customer [Name])
```

Part 11: Date Functions

TODAY()

`TODAY ()`

Returns today's date.

NOW()

`NOW ()`

Returns the current date and time.

YEAR()

`YEAR (Sales [OrderDate])`

MONTH()

`MONTH (Sales [OrderDate])`

DAY()

`DAY (Sales [OrderDate])`

Part 12: Common Interview Questions

Why are Measures better than Calculated Columns?

Answer:

- Measures are calculated at query time.
- They don't store values in memory.
- They respond dynamically to filters and slicers.

- They improve model efficiency.
-

When should you use a Calculated Column?

Use it when you need:

- A value for every row.
 - Grouping or categorization.
 - Relationships based on calculated values.
 - A column that can be used in slicers or axes.
-

Why should you use `DIVIDE()` instead of `/`?

Because `DIVIDE()` safely handles division by zero and lets you define an alternate result.

Best Practices

- Prefer **Measures** over Calculated Columns whenever possible.
 - Use `DIVIDE()` instead of `/`.
 - Use `VAR` and `RETURN` for complex measures.
 - Give measures meaningful names (e.g., `Total Sales`, `Gross Profit`).
 - Create a dedicated **Measures** table to organize all measures.
 - Avoid creating unnecessary calculated columns, as they increase model size.
-

Hands-on Practice

Using a Sales dataset, create these measures:

```
Total Sales = SUM(Sales[SalesAmount])
```

```
Total Cost = SUM(Sales[Cost])
```

```
Total Profit = [Total Sales] - [Total Cost]
```

```
Profit % = DIVIDE([Total Profit], [Total Sales], 0)
```

```
Average Sales = AVERAGE(Sales[SalesAmount])
```


Total Orders = COUNTROWS(Sales)

Unique Customers = DISTINCTCOUNT(Sales[CustomerID])

Maximum Sale = MAX(Sales[SalesAmount])

Minimum Sale = MIN(Sales[SalesAmount])

Also create:

- A calculated column for Profit = SalesAmount - Cost
- A calculated column for Order Year = YEAR(OrderDate)
- A calculated table containing only products from a chosen category using FILTER()

Homework

1. Explain the differences between Measures, Calculated Columns, and Calculated Tables with examples.
2. Create all the measures listed in the practice section.
3. Create a **Profit Category** calculated column using SWITCH(TRUE()).
4. Build a report with:
 - Total Sales
 - Total Profit
 - Profit %
 - Unique Customers
 - Average Sales
5. Compare COUNT(), COUNTA(), COUNTROWS(), and DISTINCTCOUNT() with examples.

Next Class (Class 3)

We'll cover **CALCULATE()**, **Filter Context**, **Row Context**, **Context Transition**, **FILTER()**, **ALL()**, **REMOVEFILTERS()**, **KEEPFILTERS()**, and other advanced DAX concepts that form the foundation of professional Power BI development.

Power BI Advanced Course – Class 3

Mastering CALCULATE() & Filter Context (Extreme Detail)

Duration: 3–4 Hours

Goal: By the end of this class, you will understand the most important DAX function—**CALCULATE ()**—along with **Row Context**, **Filter Context**, **Context Transition**, and filter modifier functions. These concepts are essential for writing advanced DAX.

We'll use the **Financials** sample dataset throughout the class.

Class Structure

1. What is Evaluation Context?
 2. Row Context
 3. Filter Context
 4. Context Transition
 5. CALCULATE()
 6. Boolean Filters
 7. FILTER()
 8. ALL()
 9. REMOVEFILTERS()
 10. ALLEXCEPT()
 11. KEEPFILTERS()
 12. VALUES()
 13. SELECTEDVALUE()
 14. Practical Measures
 15. Interview Questions
 16. Assignment
-

Part 1 – Evaluation Context

Every DAX calculation runs inside a **context**.

There are **2 main contexts**:

- Row Context
- Filter Context

If you don't understand context, **CALCULATE()** will always feel confusing.

Part 2 – Row Context

Definition

Row Context means DAX evaluates **one row at a time**.

Example table:

Product Sales COGS

A	100	60
B	200	120

Calculated Column:

```
Profit =  
Financials[Sales] - Financials[COGS]
```

Power BI performs:

Row 1:

$100 - 60 = 40$

Row 2:

$200 - 120 = 80$

It processes each row independently.

Important

Calculated Columns automatically have **Row Context**.

Measures **do not**.

Part 3 – Filter Context

Filter Context is created by:

- Slicers
- Filters
- Visuals
- Rows in a Matrix
- Columns in a Matrix
- Report Filters
- Page Filters

Example:

Card:

Total Sales = ₹118M

Select **Canada** in a slicer.

Now:

Total Sales = ₹25M

The measure didn't change—the **filter context** changed.

Part 4 – Difference Between Row Context & Filter Context

Row Context	Filter Context
One row at a time	Filters many rows
Mainly Calculated Columns	Mainly Measures
Created automatically in calculated columns	Created by visuals, slicers, filters

Part 5 – What is CALCULATE()?

Definition

CALCULATE() changes the **filter context** before evaluating an expression.

Syntax:

```
CALCULATE(  
    Expression,  
    Filter1,  
    Filter2,  
    ...  
)
```

Think of it as:

"Calculate this expression **under these conditions**."

Part 6 – First CALCULATE Example

Total Sales:

```
Total Sales =  
SUM(Financials[Sales])
```

Canada Sales:

```
Canada Sales =  
CALCULATE(  
    [Total Sales],  
    Financials[Country] = "Canada"  
)
```

No matter what the report shows, this measure returns only Canada's sales (unless another filter overrides it differently).

Part 7 – Multiple Filters

```
Canada Government Sales =  
CALCULATE(  
    [Total Sales],  
    Financials[Country] = "Canada",  
    Financials[Segment] = "Government"  
)
```

This applies **both** filters.

Part 8 – FILTER()

`FILTER()` creates a filtered table.

Syntax:

```
FILTER(  
    Table,  
    Condition  
)
```

Example:

```
High Sales =  
CALCULATE(  
    [Total Sales],  
    FILTER(  
        Financials,  
        Financials[Sales] > 500000  
    )  
)
```

`FILTER()` is useful when the condition cannot be expressed as a simple Boolean filter.

Part 9 – ALL()

Removes filters.

Example:

```
Total Sales All Countries =  
CALCULATE(  
    [Total Sales],  
    ALL(Financials[Country])  
)
```

Even if **Canada** is selected, this measure returns sales for **all countries**.

Part 10 – Percentage of Total

```

Sales % =
DIVIDE(
    [Total Sales],
    CALCULATE(
        [Total Sales],
        ALL(Financials)
    )
)

```

This calculates each item's contribution to the grand total.

Part 11 – REMOVEFILTERS()

Another way to remove filters.

```

Sales Ignore Country =
CALCULATE(
    [Total Sales],
    REMOVEFILTERS(Financials[Country])
)

```

Only the **Country** filter is removed.

Part 12 – ALLEXCEPT()

Removes all filters **except** the specified ones.

Example:

```

Country Total =
CALCULATE(
    [Total Sales],
    ALLEXCEPT(
        Financials,
        Financials[Country]
    )
)

```

All filters except **Country** are cleared.

Part 13 – KEEPFILTERS()

Adds a filter **without replacing** existing compatible filters.

Example:

```
Government Sales =  
CALCULATE(  
    [Total Sales],  
    KEEPFILTERS(  
        Financials[Segment] = "Government"  
    )  
)
```

This keeps existing filters and further restricts the result to the Government segment.

Part 14 – VALUES()

Returns the distinct values in a column within the current filter context.

```
VALUES(Financials[Country])
```

Useful in advanced calculations and iterators.

Part 15 – SELECTEDVALUE()

```
Selected Country =  
SELECTEDVALUE(  
    Financials[Country],  
    "Multiple Countries"  
)
```

If exactly one country is selected, it returns that country; otherwise it returns the alternate result.

Part 16 – Context Transition

One of the most important interview topics.

`CALCULATE()` converts **Row Context** into **Filter Context** when needed.

Example:

```
Customer Total =  
CALCULATE(  
    SUM(Financials[Sales])  
)
```

In calculated columns, `CALCULATE()` can perform context transition so that the current row becomes a filter context.

Part 17 – Practical Measures

Government Sales

```
Government Sales =  
CALCULATE(  
    [Total Sales],  
    Financials[Segment] = "Government"  
)
```

Enterprise Sales

```
Enterprise Sales =  
CALCULATE(  
    [Total Sales],  
    Financials[Segment] = "Enterprise"  
)
```

USA Sales

```
USA Sales =  
CALCULATE(  
    [Total Sales],  
    Financials[Country] = "United States of America"  
)
```

High Discount Sales

```
High Discount Sales =  
CALCULATE(  
    [Total Sales],  
    Financials[Discount Band] = "High"  
)
```

Total Sales Ignoring Country

```
Total Sales Ignore Country =  
CALCULATE(  
    [Total Sales],  
    ALL(Financials[Country])  
)
```

Sales by Selected Country

```
Sales by Selected Country =  
VAR SelectedCountry =  
    SELECTEDVALUE(Financials[Country])  
  
RETURN  
CALCULATE(  
    [Total Sales],  
    Financials[Country] = SelectedCountry  
)
```

Part 18 – Visual Practice

Create:

1. **Cards**
 - Total Sales
 - Government Sales
 - USA Sales
2. **Matrix**
 - Rows: Country
 - Columns: Product
 - Values:
 - Total Sales
 - Sales %
3. **Bar Chart**
 - Country vs Total Sales
4. **Pie Chart**
 - Segment vs Sales
5. **Slicers**
 - Country
 - Segment
 - Product
 - Discount Band

Observe how each measure behaves as you change slicer selections.

Common Interview Questions

1. What is `CALCULATE ()` ?

Answer: It evaluates an expression after modifying the filter context.

2. Which is the most important DAX function?

Answer: `CALCULATE ()`, because many advanced DAX calculations are built around it.

3. What is Filter Context?

Answer: The set of filters applied to a calculation by visuals, slicers, page filters, report filters, or DAX.

4. What is Row Context?

Answer: The context where DAX evaluates one row at a time, typically in calculated columns and iterator functions.

5. Difference between `ALL ()` and `REMOVEFILTERS ()` ?

- `ALL ()` removes filters and also returns a table, so it can be used in table expressions.
 - `REMOVEFILTERS ()` is intended specifically to clear filters in `CALCULATE ()` and similar contexts.
-

Assignment

Using the **Financials** sample dataset, create these measures:

1. Total Sales
2. Canada Sales
3. France Sales

4. Government Sales
 5. Enterprise Sales
 6. High Discount Sales
 7. Sales ignoring Country (`ALL`)
 8. Sales ignoring Product (`REMOVEFILTERS`)
 9. Sales by Country (`ALLEXCEPT`)
 10. Selected Country (`SELECTEDVALUE`)
 11. Sales % of Total
 12. Profit % of Total
-

Homework Challenge

Without referring to notes, write measures for:

- Sales for Germany only
- Sales for the Small Business segment
- Sales ignoring the Country filter but respecting Product
- Profit % of Grand Total
- A dynamic title that displays the selected country using `SELECTEDVALUE()`

Mastering these concepts is the key to understanding advanced DAX. Once you're comfortable with `CALCULATE()` and filter context, you'll be ready for **Class 4**, where you'll learn **Time Intelligence** functions such as **YTD, QTD, MTD, SAMEPERIODLASTYEAR, DATEADD, YOY, MOM, Running Total, Rolling 12 Months**, and **dynamic date calculations** in depth.

Power BI Advanced Course – Class 4

Time Intelligence in DAX (Extreme Detail)

Duration: 3–4 Hours

Goal: By the end of this class, you'll be able to build professional business KPIs like **YTD**, **QTD**, **MTD**, **YOY**, **YOY%**, **MOM**, **MOM%**, **Running Total**, **Rolling 12 Months**, **Previous Month**, **Previous Year**, and understand when to use each.

We will use the **Financials** sample dataset.

Agenda

1. What is Time Intelligence?
 2. Date Table
 3. Why Mark as Date Table?
 4. Calendar Functions
 5. MTD
 6. QTD
 7. YTD
 8. Previous Month
 9. Previous Year
 10. DATEADD()
 11. SAMEPERIODLASTYEAR()
 12. YOY
 13. YOY %
 14. MOM
 15. MOM %
 16. Running Total
 17. Rolling 12 Months
 18. Dynamic Time KPIs
 19. Dashboard Practice
 20. Interview Questions
 21. Assignment
-

Part 1 – What is Time Intelligence?

Time Intelligence means comparing values across time.

Examples:

- Sales This Month
- Sales Last Month
- Sales This Year
- Sales Last Year
- YTD Sales
- Growth %
- Running Total

Business managers use these every day.

Part 2 – Date Table (Mandatory)

Time Intelligence requires a proper Date Table.

Your Financials table has a `Date` column, but you should create a dedicated Date table.

Create Date Table

```
Date =  
CALENDAR(  
    MIN(Financials[Date]),  
    MAX(Financials[Date])  
)
```

Add Year

```
Year =  
YEAR(Date[Date])
```

Month Number

```
Month No =  
MONTH(Date[Date])
```

Month Name

```
Month Name =  
FORMAT(Date[Date], "MMMM")
```

Quarter

```
Quarter =  
"Q" & FORMAT(Date[Date], "Q")
```

Year Month

```
Year Month =  
FORMAT(Date[Date], "YYYY-MM")
```

Mark as Date Table

Table Tools → **Mark as Date Table**

Choose:

```
Date[Date]
```

Part 3 – Create Base Measure

Everything starts with:

```
Total Sales =  
SUM(Financials[Sales])
```

Part 4 – MTD (Month-To-Date)

Meaning:

Current month's sales from Day 1 until today.

Example

July

Date Sales

1	₹100
2	₹200

Date Sales

3 ₹300

MTD on 3 July:

$$100+200+300 = 600$$

Formula

```
MTD Sales =  
TOTALMTD(  
    [Total Sales],  
    Date[Date]  
)
```

Part 5 – QTD (Quarter-To-Date)

Current quarter sales.

Example

Quarter 1

January

February

March

QTD in March:

$$\text{Jan} + \text{Feb} + \text{Mar}$$

Formula

```
QTD Sales =  
TOTALQTD(  
    [Total Sales],  
    Date[Date]  
)
```

Part 6 – YTD (Year-To-Date)

Current year's sales until today's date.

Example

January

February

March

April

YTD in April:

Jan + Feb + Mar + Apr

Formula

```
YTD Sales =  
TOTALYTD(  
    [Total Sales],  
    Date[Date]  
)
```

Part 7 – DATESYTD()

Another method.

```
YTD Sales =  
CALCULATE(  
    [Total Sales],  
    DATESYTD(Date[Date])  
)
```

Same result.

Part 8 – Previous Month

```
Previous Month Sales =  
CALCULATE(  
    [Total Sales],  
    PREVIOUSMONTH(Date[Date])  
)
```

Part 9 – Previous Year

```
Previous Year Sales =  
CALCULATE(  
    [Total Sales],  
    PREVIOUSYEAR(Date[Date])  
)
```

Part 10 – DATEADD()

Moves dates forward or backward.

Previous Month

```
Sales Last Month =  
CALCULATE(  
    [Total Sales],  
    DATEADD(  
        Date[Date],  
        -1,  
        MONTH  
    )  
)
```

Previous Year

```
Sales Last Year =  
CALCULATE(  
    [Total Sales],  
    DATEADD(  
        Date[Date],  
        -1,  
        YEAR  
    )  
)
```

Next Month

```
DATEADD(  
    Date[Date],  
    1,  
    MONTH  
)
```

Part 11 – SAMEPERIODLASTYEAR()

One of the most used functions.

```
Last Year Sales =  
CALCULATE(  
    [Total Sales],  
    SAMEPERIODLASTYEAR(  
        Date[Date]  
    )  
)
```

If today is:

March 2024

Returns

March 2023

Part 12 – YOY (Year-over-Year)

Difference between current year and last year.

```
YOY Sales =  
[Total Sales]  
-  
[Last Year Sales]
```

Example

Year Sales

2023 ₹100

2024 ₹120

YOY

120-100=20

Part 13 – YOY %

```
YOY % =  
DIVIDE(  
    [YOY Sales],  
    [Last Year Sales],  
    0  
)
```

Example

Current

₹120

Previous

₹100

Growth

20%

Part 14 – MOM (Month-over-Month)

Previous Month

```
Last Month Sales =  
CALCULATE(  
    [Total Sales],  
    DATEADD(  
        Date[Date],  
        -1,  
        MONTH  
    )  
)
```

Difference

```
MOM =  
[Total Sales]  
-  
[Last Month Sales]
```

Part 15 – MOM %

```
MOM % =  
DIVIDE(  
    [MOM],  
    [Last Month Sales],  
    0  
)
```

Part 16 – Running Total

Very common interview question.

```
Running Total =  
CALCULATE(  
    [Total Sales],  
    FILTER(  
        ALL(Date),  
        Date[Date]  
        <= MAX(Date[Date])  
    )  
)
```

Example

Month	Sales	Running Total
Jan	100	100
Feb	200	300
Mar	300	600
Apr	400	1000

Part 17 – Rolling 12 Months

```
Rolling 12 Months =  
CALCULATE(  
    [Total Sales],  
    DATESINPERIOD(  
        Date[Date],  
        MAX(Date[Date]),  
        -12,  
        MONTH  
    )  
)
```

Returns sales for the last 12 months relative to the current date context.

Part 18 – Dynamic KPI Card

Selected Year

```
Selected Year =  
SELECTEDVALUE(
```

```
Date[Year],  
"All Years"  
)
```

Dynamic Title

```
Sales Title =  
"Sales Report - "  
&  
SELECTEDVALUE(  
Date[Year],  
"All Years"  
)
```

Part 19 – Dashboard

Create these KPI Cards:

- Total Sales
 - YTD Sales
 - MTD Sales
 - QTD Sales
 - Last Year Sales
 - YOY %
 - MOM %
 - Running Total
-

Charts

Line Chart

Axis

Month Name

Values

```
Total Sales  
Running Total
```

Clustered Column

Axis

Year

Values

Total Sales
Last Year Sales

Matrix

Rows

Country

Columns

Year

Values

YTD
YOY%

Slicers

- Year
 - Country
 - Product
 - Segment
-

Part 20 – Interview Questions

Difference between YTD and Running Total?

YTD

- Resets every year.
- Calculates from the start of the year to the current date.

Running Total

- Continues accumulating according to the current date context.
- Can span multiple years unless you intentionally reset it.

Difference between DATEADD() and SAMEPERIODLASTYEAR()?

DATEADD()

- Flexible.
- Can shift by days, months, quarters, or years.

SAMEPERIODLASTYEAR()

- Specifically compares the same period one year earlier.

Which function is best for YOY?

- `SAMEPERIODLASTYEAR()`
- Or `DATEADD(..., -1, YEAR)`

Both are common, but `SAMEPERIODLASTYEAR()` is often simpler for standard year-over-year comparisons.

Assignment

Create these measures:

1. Total Sales
2. MTD Sales
3. QTD Sales
4. YTD Sales
5. Last Month Sales
6. Previous Year Sales
7. Last Year Sales
8. YOY Sales
9. YOY %
10. MOM
11. MOM %
12. Running Total
13. Rolling 12 Months
14. Dynamic Sales Title

Homework Challenge

Using the **Financials** sample dataset:

1. Create a dashboard with:
 - 8 KPI cards
 - 2 Line Charts
 - 1 Matrix
 - 4 Slicers
2. Format:
 - YOY % and MOM % as percentages.
 - Currency measures as ₹ or your preferred currency.
3. Add conditional formatting:
 - Green for positive growth.
 - Red for negative growth.

After completing this class, you'll be able to build most of the time-based KPIs used in real-world Power BI dashboards. The next step is **Class 5**, where you'll learn **advanced DAX functions** like SUMX, RANKX, RELATED, RELATEDTABLE, LOOKUPVALUE, SUMMARIZE, ADDCOLUMNS, TOPN, virtual tables, and other techniques used in enterprise reporting.